

SQL Data Analysis

Data Analytics — Project 3

Intern Name:	Maleeha Fatima	Date:	30th may, 2026	Tool Used:	DB Browser for SQLite
---------------------	----------------	--------------	----------------	-------------------	-----------------------

Dataset Overview

Dataset: Online Orders Data

Columns: OrderID, Date, CustomerID, Product, Quantity, UnitPrice, ShippingAddress, PaymentMethod, OrderStatus, TrackingNumber, ItemsInCart, CouponCode, ReferralSource, TotalPrice

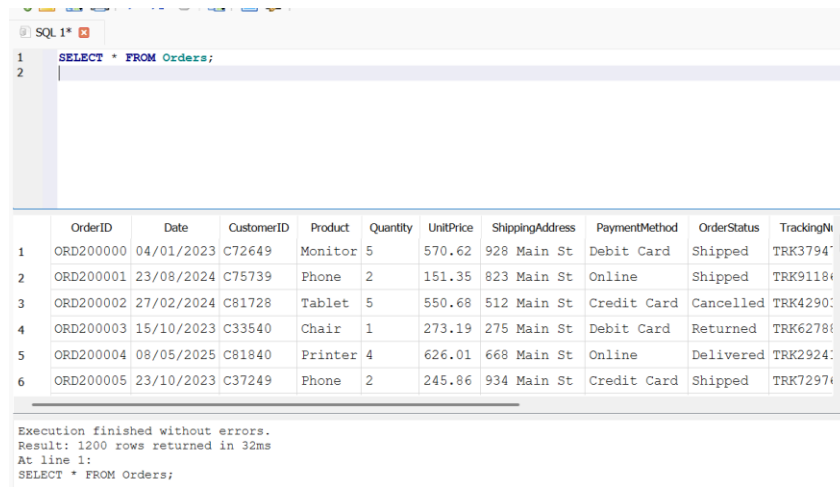
SELECT Queries

Goal: Retrieve data from the Orders table using SELECT statements.

Query 1: View all orders

This query retrieves every row and column from the Orders table to get a full overview of the dataset.

```
-- Query 1: View all orders
SELECT * FROM Orders;
```



	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingN
1	ORD200000	04/01/2023	C72649	Monitor	5	570.62	928 Main St	Debit Card	Shipped	TRK3794
2	ORD200001	23/08/2024	C75739	Phone	2	151.35	823 Main St	Online	Shipped	TRK91186
3	ORD200002	27/02/2024	C81728	Tablet	5	550.68	512 Main St	Credit Card	Cancelled	TRK42900
4	ORD200003	15/10/2023	C33540	Chair	1	273.19	275 Main St	Debit Card	Returned	TRK62786
5	ORD200004	08/05/2025	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29240
6	ORD200005	23/10/2023	C37249	Phone	2	245.86	934 Main St	Credit Card	Shipped	TRK72976

Execution finished without errors.
Result: 1200 rows returned in 32ms
At line 1:
SELECT * FROM Orders;

Query 2: View key columns only

This query selects only the most important columns to focus on what was ordered and at what price.

```
-- Query 2: View key columns only
SELECT OrderID, Product, Quantity, TotalPrice FROM Orders;
```

SQL 1* 

```
1 SELECT * FROM Orders;
2 SELECT OrderID, Product, Quantity, TotalPrice FROM Orders;
```

	OrderID	Product	Quantity	TotalPrice	
1	ORD200000	Monitor	5	2,853.10	
2	ORD200001	Phone	2	302.7	
3	ORD200002	Tablet	5	2,753.40	
4	ORD200003	Chair	1	273.19	
5	ORD200004	Printer	4	2,504.04	
6	ORD200005	Phone	2	491.72	
7	ORD200006	Laptop	1	664.42	

Execution finished without errors.
Result: 1200 rows returned in 29ms
At line 2:
SELECT OrderID, Product, Quantity, TotalPrice FROM Orders;

Query 3: View all unique products

This query shows all the different products available in the dataset without repetition.

```
-- Query 3: View all unique products
SELECT DISTINCT Product FROM Orders;
```

SQL 1* ✕	
1	SELECT * FROM Orders;
2	SELECT OrderID, Product, Quantity, TotalPrice FROM Orders;
3	SELECT DISTINCT Product FROM Orders;
	Product
1	Monitor
2	Phone
3	Tablet
4	Chair
5	Printer
6	Laptop
7	Desk
Execution finished without errors. Result: 7 rows returned in 11ms At line 3: SELECT DISTINCT Product FROM Orders;	

Query 4: View all unique payment methods

This query shows all the different payment methods customers used to place orders.

```
-- Query 4: View all unique payment methods
SELECT DISTINCT PaymentMethod FROM Orders;
```

SQL 1* 	
1	SELECT * FROM Orders;
2	SELECT OrderID, Product, Quantity, TotalPrice FROM Orders;
3	SELECT DISTINCT Product FROM Orders;
4	SELECT DISTINCT PaymentMethod FROM Orders;

	PaymentMethod
1	Debit Card
2	Online
3	Credit Card
4	Gift Card
5	Cash

Execution finished without errors.
Result: 5 rows returned in 15ms
At line 4:
SELECT DISTINCT PaymentMethod FROM Orders;

WHERE Clause (Filtering)

Goal: Filter rows based on specific conditions to isolate meaningful data.

Query 5: Orders that were delivered

This query filters only the orders with a Delivered status to see successfully completed orders.

```
-- Query 5: Orders that were delivered
SELECT * FROM Orders WHERE OrderStatus = 'Delivered';
```

SQL 1*										
1	SELECT * FROM Orders;									
2	SELECT OrderID, Product, Quantity, TotalPrice FROM Orders;									
3	SELECT DISTINCT Product FROM Orders;									
4	SELECT DISTINCT PaymentMethod FROM Orders;									
5	SELECT * FROM Orders WHERE OrderStatus = 'Delivered';									
	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber
1	ORD200004	08/05/2025	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29241.
2	ORD200015	17/07/2023	C39416	Printer	1	473.96	942 Main St	Cash	Delivered	TRK54930'
3	ORD200021	17/11/2024	C99023	Monitor	4	342.95	569 Main St	Cash	Delivered	TRK52646'
4	ORD200024	30/10/2024	C17470	Laptop	2	230.95	956 Main St	Online	Delivered	TRK28775'
5	ORD200025	24/07/2024	C42577	Monitor	1	399.98	709 Main St	Debit Card	Delivered	TRK69423'
6	ORD200026	06/03/2024	C21944	Laptop	5	59.55	296 Main St	Online	Delivered	TRK80276'
Execution finished without errors. Result: 231 rows returned in 38ms At line 5: SELECT * FROM Orders WHERE OrderStatus = 'Delivered';										

Query 6: Orders with TotalPrice above 1000

This query finds high-value orders where the total price exceeds 1000 to identify big purchases.

```
-- Query 6: High value orders above 1000
SELECT OrderID, Product, TotalPrice FROM Orders WHERE TotalPrice > 1000;
```

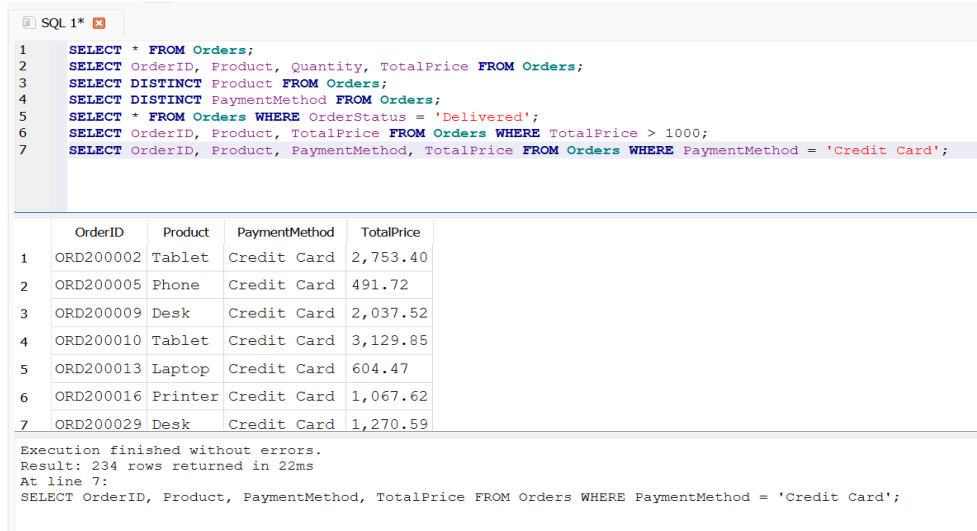
SQL 1*			
1	SELECT * FROM Orders;		
2	SELECT OrderID, Product, Quantity, TotalPrice FROM Orders;		
3	SELECT DISTINCT Product FROM Orders;		
4	SELECT DISTINCT PaymentMethod FROM Orders;		
5	SELECT * FROM Orders WHERE OrderStatus = 'Delivered';		
6	SELECT OrderID, Product, TotalPrice FROM Orders WHERE TotalPrice > 1000;		
	OrderID	Product	TotalPrice
1	ORD200000	Monitor	2,853.10
2	ORD200002	Tablet	2,753.40
3	ORD200004	Printer	2,504.04
4	ORD200009	Desk	2,037.52
5	ORD200010	Tablet	3,129.85
6	ORD200016	Printer	1,067.62
7	ORD200019	Monitor	1,120.00
Execution finished without errors. Result: 512 rows returned in 16ms At line 6: SELECT OrderID, Product, TotalPrice FROM Orders WHERE TotalPrice > 1000;			

Query 7: Orders paid by Credit Card

This query isolates all orders paid using a Credit Card to analyse that payment segment.

-- Query 7: Credit card orders

```
SELECT OrderID, Product, PaymentMethod, TotalPrice FROM Orders WHERE PaymentMethod = 'Credit Card';
```



```
SQL 1*
```

```
1 SELECT * FROM Orders;
2 SELECT OrderID, Product, Quantity, TotalPrice FROM Orders;
3 SELECT DISTINCT Product FROM Orders;
4 SELECT DISTINCT PaymentMethod FROM Orders;
5 SELECT * FROM Orders WHERE OrderStatus = 'Delivered';
6 SELECT OrderID, Product, TotalPrice FROM Orders WHERE TotalPrice > 1000;
7 SELECT OrderID, Product, PaymentMethod, TotalPrice FROM Orders WHERE PaymentMethod = 'Credit Card';
```

	OrderID	Product	PaymentMethod	TotalPrice
1	ORD200002	Tablet	Credit Card	2,753.40
2	ORD200005	Phone	Credit Card	491.72
3	ORD200009	Desk	Credit Card	2,037.52
4	ORD200010	Tablet	Credit Card	3,129.85
5	ORD200013	Laptop	Credit Card	604.47
6	ORD200016	Printer	Credit Card	1,067.62
7	ORD200029	Desk	Credit Card	1,270.59

Execution finished without errors.
Result: 234 rows returned in 22ms
At line 7:
SELECT OrderID, Product, PaymentMethod, TotalPrice FROM Orders WHERE PaymentMethod = 'Credit Card';

Query 8: Cancelled or Returned orders

This query finds all problematic orders that were either cancelled or returned to assess order issues.

-- Query 8: Cancelled or returned orders

```
SELECT OrderID, Product, OrderStatus FROM Orders WHERE OrderStatus = 'Cancelled' OR OrderStatus = 'Returned';
```



```
SQL 1*
```

```
1 SELECT * FROM Orders;
2 SELECT OrderID, Product, Quantity, TotalPrice FROM Orders;
3 SELECT DISTINCT Product FROM Orders;
4 SELECT DISTINCT PaymentMethod FROM Orders;
5 SELECT * FROM Orders WHERE OrderStatus = 'Delivered';
6 SELECT OrderID, Product, TotalPrice FROM Orders WHERE TotalPrice > 1000;
7 SELECT OrderID, Product, PaymentMethod, TotalPrice FROM Orders WHERE PaymentMethod = 'Credit Card';
8 SELECT OrderID, Product, OrderStatus FROM Orders WHERE OrderStatus = 'Cancelled' OR OrderStatus = 'Returned';
9
```

	OrderID	Product	OrderStatus
1	ORD200002	Tablet	Cancelled
2	ORD200003	Chair	Returned
3	ORD200006	Laptop	Returned
4	ORD200008	Phone	Cancelled
5	ORD200010	Tablet	Returned
6	ORD200011	Monitor	Returned
7	ORD200013	Laptop	Returned

Execution finished without errors.
Result: 497 rows returned in 24ms
At line 8:
SELECT OrderID, Product, OrderStatus FROM Orders WHERE OrderStatus = 'Cancelled' OR OrderStatus = 'Returned';

ORDER BY (Sorting)

Goal: Sort query results in ascending or descending order.

Query 9: Orders sorted by TotalPrice (highest first)

This query sorts all orders from the highest to lowest total price to identify the most valuable orders.

```
-- Query 9: Orders sorted by highest price first
SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC;
```

```
SQL 1*
1 SELECT * FROM Orders;
2 SELECT OrderID, Product, Quantity, TotalPrice FROM Orders;
3 SELECT DISTINCT Product FROM Orders;
4 SELECT DISTINCT PaymentMethod FROM Orders;
5 SELECT * FROM Orders WHERE OrderStatus = 'Delivered';
6 SELECT OrderID, Product, TotalPrice FROM Orders WHERE TotalPrice > 1000;
7 SELECT OrderID, Product, PaymentMethod, TotalPrice FROM Orders WHERE PaymentMethod = 'Credit Card';
8 SELECT OrderID, Product, OrderStatus FROM Orders WHERE OrderStatus = 'Cancelled' OR OrderStatus = 'Returned';
9 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC;
10
```

	OrderID	Product	TotalPrice
1	ORD200789	Tablet	3,456.40
2	ORD201122	Monitor	3,390.95
3	ORD200632	Laptop	3,390.80
4	ORD200469	Chair	3,384.90
5	ORD200328	Tablet	3,370.20
6	ORD200107	Printer	3,353.75
7	ORD200326	Laptop	3,352.40

Execution finished without errors.
Result: 1200 rows returned in 16ms
At line 9:
SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC;

Query 10: Top 10 most expensive orders

This query retrieves only the top 10 highest value orders, useful for spotting premium customers.

```
-- Query 10: Top 10 most expensive orders
SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC LIMIT 10;
```

```
SQL1*
```

```
3 SELECT DISTINCT Product FROM Orders;
4 SELECT DISTINCT PaymentMethod FROM Orders;
5 SELECT * FROM Orders WHERE OrderStatus = 'Delivered';
6 SELECT OrderID, Product, TotalPrice FROM Orders WHERE TotalPrice > 1000;
7 SELECT OrderID, Product, PaymentMethod, TotalPrice FROM Orders WHERE PaymentMethod = 'Credit Card';
8 SELECT OrderID, Product, OrderStatus FROM Orders WHERE OrderStatus = 'Cancelled' OR OrderStatus = 'Returned';
9 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC;
10 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC LIMIT 10;
11
```

	OrderID	Product	TotalPrice
1	ORD200789	Tablet	3,456.40
2	ORD201122	Monitor	3,390.95
3	ORD200632	Laptop	3,390.80
4	ORD200469	Chair	3,384.90
5	ORD200328	Tablet	3,370.20
6	ORD200107	Printer	3,353.75
7	ORD200326	Laptop	3,352.40

Execution finished without errors.
Result: 10 rows returned in 16ms
At line 10:
SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC LIMIT 10;

Query 11: Orders sorted alphabetically by product

This query sorts orders alphabetically by product name to make it easier to find specific products.

```
-- Query 11: Orders sorted alphabetically by product
SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY Product ASC;
```

SQL 1*

```
3  SELECT DISTINCT Product FROM Orders;
4  SELECT DISTINCT PaymentMethod FROM Orders;
5  SELECT * FROM Orders WHERE OrderStatus = 'Delivered';
6  SELECT OrderID, Product, TotalPrice FROM Orders WHERE TotalPrice > 1000;
7  SELECT OrderID, Product, PaymentMethod, TotalPrice FROM Orders WHERE PaymentMethod = 'Credit Card';
8  SELECT OrderID, Product, OrderStatus FROM Orders WHERE OrderStatus = 'Cancelled' OR OrderStatus = 'Returned';
9  SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC;
10 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC LIMIT 10;
11 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY Product ASC;
```

	OrderID	Product	TotalPrice
1	ORD200003	Chair	273.19
2	ORD200020	Chair	312.55
3	ORD200031	Chair	1,500.24
4	ORD200033	Chair	162.77
5	ORD200034	Chair	2,306.08
6	ORD200043	Chair	1,497.63
7	ORD200045	Chair	317.25

Execution finished without errors.

Result: 1200 rows returned in 12ms

At line 11:

```
SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY Product ASC;
```

GROUP BY + Aggregations

Goal: Group data into categories and calculate COUNT, SUM, and AVG for business insights.

Query 12: Count of orders per product

This query counts how many times each product was ordered to identify the most popular products.

-- Query 12: Count of orders per product

```
SELECT Product, COUNT(*) AS total_orders FROM Orders GROUP BY Product ORDER BY total_orders DESC;
```

```
6 SELECT OrderID, Product, TotalPrice FROM Orders WHERE TotalPrice > 1000;
7 SELECT OrderID, Product, PaymentMethod, TotalPrice FROM Orders WHERE PaymentMethod = 'Credit Card';
8 SELECT OrderID, Product, OrderStatus FROM Orders WHERE OrderStatus = 'Cancelled' OR OrderStatus = 'Returned';
9 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC;
10 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC LIMIT 10;
11 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY Product ASC;
12 SELECT Product, COUNT(*) AS total_orders FROM Orders GROUP BY Product ORDER BY total_orders DESC;
```

	Product	total_orders
1	Printer	181
2	Tablet	179
3	Chair	178
4	Laptop	173
5	Desk	170
6	Monitor	163
7	Phone	156

Execution finished without errors.
Result: 7 rows returned in 13ms
At line 12:
SELECT Product, COUNT(*) AS total_orders FROM Orders GROUP BY Product ORDER BY total_orders DESC;

Query 13: Total revenue per product

This query calculates the total revenue generated by each product to find the highest earning products.

-- Query 13: Total revenue per product

```
SELECT Product, SUM(TotalPrice) AS total_revenue FROM Orders GROUP BY Product ORDER BY total_revenue DESC;
```

```
6 SELECT OrderID, Product, TotalPrice FROM Orders WHERE TotalPrice > 1000;
7 SELECT OrderID, Product, PaymentMethod, TotalPrice FROM Orders WHERE PaymentMethod = 'Credit Card';
8 SELECT OrderID, Product, OrderStatus FROM Orders WHERE OrderStatus = 'Cancelled' OR OrderStatus = 'Returned';
9 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC;
10 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC LIMIT 10;
11 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY Product ASC;
12 SELECT Product, COUNT(*) AS total_orders FROM Orders GROUP BY Product ORDER BY total_orders DESC;
13 SELECT Product, SUM(TotalPrice) AS total_revenue FROM Orders GROUP BY Product ORDER BY total_revenue DESC;
```

	Product	total_revenue
1	Phone	54211.88
2	Tablet	49889.77
3	Laptop	47415.2
4	Printer	46393.92
5	Desk	43318.99
6	Chair	42676.39
7	Monitor	38021.74

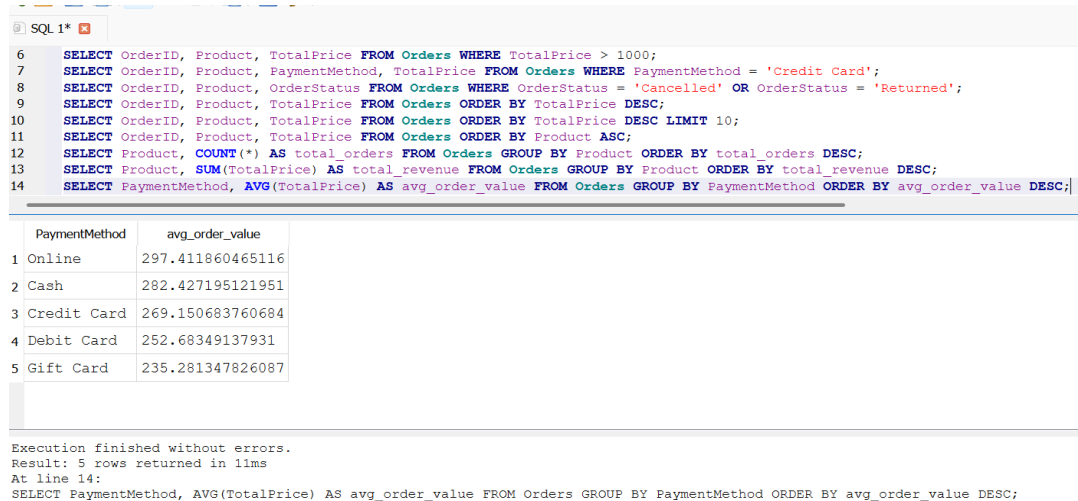
Execution finished without errors.
Result: 7 rows returned in 18ms
At line 13:
SELECT Product, SUM(TotalPrice) AS total_revenue FROM Orders GROUP BY Product ORDER BY total_revenue DESC;

Query 14: Average order value per payment method

This query calculates the average order value for each payment method to understand spending behaviour by payment type.

```
-- Query 14: Average order value per payment method
```

```
SELECT PaymentMethod, AVG(TotalPrice) AS avg_order_value FROM Orders GROUP BY PaymentMethod ORDER BY avg_order_value DESC;
```



The screenshot shows a SQL query editor with a query that includes several clauses: WHERE TotalPrice > 1000, WHERE PaymentMethod = 'Credit Card', WHERE OrderStatus = 'Cancelled' OR OrderStatus = 'Returned', ORDER BY TotalPrice DESC, ORDER BY TotalPrice DESC LIMIT 10, ORDER BY Product ASC, GROUP BY Product, and GROUP BY PaymentMethod. The results table shows the average order value for five payment methods: Online (297.411860465116), Cash (282.427195121951), Credit Card (269.150683760684), Debit Card (252.68349137931), and Gift Card (235.281347826087).

	PaymentMethod	avg_order_value
1	Online	297.411860465116
2	Cash	282.427195121951
3	Credit Card	269.150683760684
4	Debit Card	252.68349137931
5	Gift Card	235.281347826087

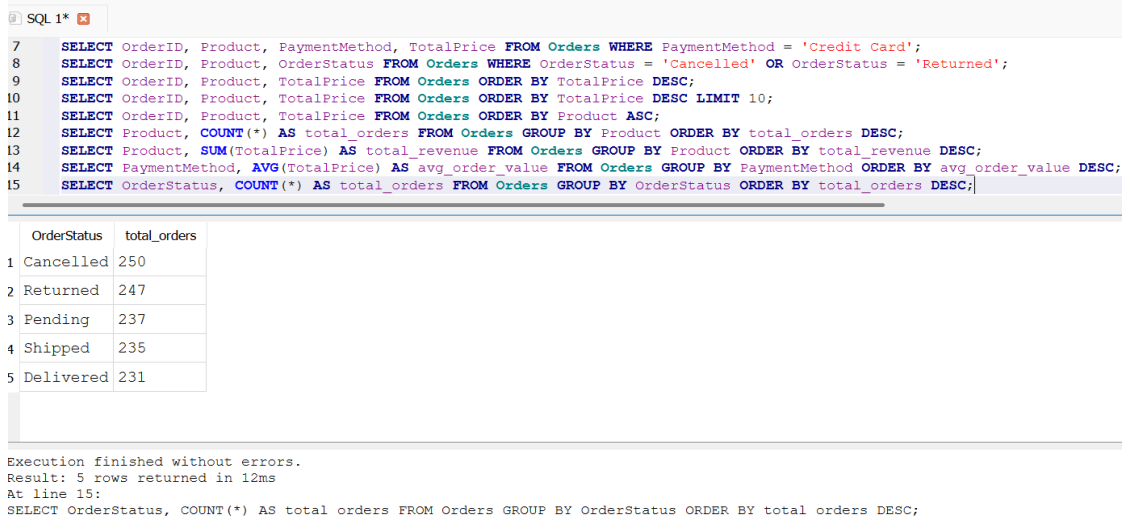
Execution finished without errors.
Result: 5 rows returned in 11ms
At line 14:
SELECT PaymentMethod, AVG(TotalPrice) AS avg_order_value FROM Orders GROUP BY PaymentMethod ORDER BY avg_order_value DESC;

Query 15: Count of orders per order status

This query groups orders by their status to see how many were delivered, shipped, cancelled, or returned.

```
-- Query 15: Count of orders per status
```

```
SELECT OrderStatus, COUNT(*) AS total_orders FROM Orders GROUP BY OrderStatus ORDER BY total_orders DESC;
```



The screenshot shows a SQL query editor with a query that includes several clauses: WHERE PaymentMethod = 'Credit Card', WHERE OrderStatus = 'Cancelled' OR OrderStatus = 'Returned', ORDER BY TotalPrice DESC, ORDER BY TotalPrice DESC LIMIT 10, ORDER BY Product ASC, GROUP BY Product, and GROUP BY OrderStatus. The results table shows the count of orders for five order statuses: Cancelled (250), Returned (247), Pending (237), Shipped (235), and Delivered (231).

	OrderStatus	total_orders
1	Cancelled	250
2	Returned	247
3	Pending	237
4	Shipped	235
5	Delivered	231

Execution finished without errors.
Result: 5 rows returned in 12ms
At line 15:
SELECT OrderStatus, COUNT(*) AS total_orders FROM Orders GROUP BY OrderStatus ORDER BY total_orders DESC;

Query 16 (Bonus): Products with more than 50 orders — HAVING clause

This bonus query uses HAVING to filter grouped results, showing only products that received more than 50 orders. HAVING filters groups, unlike WHERE which filters individual rows.

```
-- Query 16 (Bonus): Products with more than 50 orders
```

```
SELECT Product, COUNT(*) AS total_orders FROM Orders GROUP BY Product HAVING total_orders > 50  
ORDER BY total_orders DESC;
```

```
SQL 1* x  
8 SELECT OrderID, Product, OrderStatus FROM Orders WHERE OrderStatus = 'Cancelled' OR OrderStatus = 'Returned';  
9 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC;  
10 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY TotalPrice DESC LIMIT 10;  
11 SELECT OrderID, Product, TotalPrice FROM Orders ORDER BY Product ASC;  
12 SELECT Product, COUNT(*) AS total_orders FROM Orders GROUP BY Product ORDER BY total_orders DESC;  
13 SELECT Product, SUM(TotalPrice) AS total_revenue FROM Orders GROUP BY Product ORDER BY total_revenue DESC;  
14 SELECT PaymentMethod, AVG(TotalPrice) AS avg_order_value FROM Orders GROUP BY PaymentMethod ORDER BY avg_order_value DESC;  
15 SELECT OrderStatus, COUNT(*) AS total_orders FROM Orders GROUP BY OrderStatus ORDER BY total_orders DESC;  
16 SELECT Product, COUNT(*) AS total_orders FROM Orders GROUP BY Product HAVING total_orders > 50 ORDER BY total_orders DESC;
```

	Product	total_orders
1	Printer	181
2	Tablet	179
3	Chair	178
4	Laptop	173
5	Desk	170
6	Monitor	163
7	Phone	156

Execution finished without errors.

Result: 7 rows returned in 17ms

At line 16:

```
SELECT Product, COUNT(*) AS total_orders FROM Orders GROUP BY Product HAVING total_orders > 50 ORDER BY total_orders DESC;
```